

# 표준 기술 문서 (Standard Technical Document)

## 프로젝트명: 톡참새 (EnglishYoutube)

### 1. 개요 (Overview)

본 문서는 "톡참새 (EnglishYoutube)" 프로젝트의 아키텍처, 기술 스택, 배포 워크플로우 및 보안 설정에 대한 표준 기술 지침을 제공합니다. 본 프로젝트는 Next.js 프레임워크를 기반으로 구축된 웹 애플리케이션으로, 도커(Docker) 컨테이너 기반으로 완벽하게 가상화되어 로컬 개발 환경부터 운영 서버까지 일관된 실행 환경을 보장합니다.

### 2. 기술 스택 (Tech Stack)

- Frontend/Backend: Next.js
- Containerization: Docker, Docker Compose
- Web Server / Reverse Proxy: Nginx
- OS (Target): Rocky Linux VM (운영 환경), Windows (개발 환경)

### 3. 시스템 아키텍처 (System Architecture)

- 외부 요청 처리: 클라이언트의 HTTP 요청(포트 80)은 운영 서버의 Nginx 리버스 프록시가 수신합니다.
- 내부 라우팅: Nginx 는 수신한 요청을 Docker 내부 네트워크를 통해 격리된 Next.js 앱 컨테이너(포트 3000)로 전달합니다.
- 격리 및 보안: Next.js 애플리케이션은 외부에 직접 노출되지 않으며(외부 포트 3000 다이렉트 접속 차단), Nginx 를 거쳐야만 접근 가능합니다.

### 4. 배포 워크플로우 (Deployment Workflow)

배포 프로세스는 스크립트를 통해 자동화되어 있으며, 역할에 따라 구분됩니다.

#### 4.1 개발자 환경 (Windows Host PC)

- build.bat: 코드 수정 후, 로컬 PC 에서 Docker 기반으로 컨테이너를 빌드하고 실행하여 정상 작동 여부를 테스트합니다. (<http://localhost> 접속)
- docker-push.ps1: 개발 완료 후 새로운 릴리스 버전을 빌드하고, 생성된 도커 이미지를 Docker Hub (yslee4050/talkchamsae) 레지스트리에 업로드합니다.

#### 4.2 운영 환경 (Rocky Linux VM)

- deploy.sh: Docker Hub 에 업로드된 최신 이미지를 다운로드받아, 무중단/고속으로 컨테이너를 재시작합니다. 빌드 과정을 생략하므로 배포 속도가 매우 빠릅니다. 80 번 포트에 대한 방화벽 설정도 자동으로 처리합니다.
- build.sh: 예외적으로 리눅스 환경에서 직접 소스 코드를 빌드해야 할 경우 사용됩니다.

### 5. 네트워크 포트 및 보안 규정 (Network & Security)

운영 서버(VM) 기준 네트워크 포트 접근 제어 정책은 다음과 같습니다.

| 목적지 포트 | 서비스           | 외부 접속  | 내부(Docker) 접속 | 비고                                                                                |
|--------|---------------|--------|---------------|-----------------------------------------------------------------------------------|
| 80     | Nginx         | 허용 (O) | 허용 (O)        | 웹 서비스<br>출입구.<br><a href="http://&lt;VM_IP&gt;">http://&lt;VM_IP&gt;</a><br>요청 수신 |
| 3000   | Next.js (web) | 차단 (X) | 허용 (O)        | 외부 다이렉트<br>접속 불가,<br>Nginx 를<br>통해서만 통신                                           |
| 22     | SSH           | 허용 (O) | -             | 서버 관리자용<br>가상머신 접속<br>포트                                                          |

### 6. 결론 (Conclusion)

본 프로젝트는 Docker 를 활용하여 개발/운영 환경 간의 파편화를 제거하였으며, Nginx 리버스 프록시를 도입하여 보안성과 확장성을 동시에 확보했습니다. 자동화된 스크립트를 통해 신속하고 안정적인 CI/CD 및 인프라 관리를 지원합니다.